

Лабораторная работа № 4

Имитационная модель TCP/Reno и активного управления очередью RED

Куокконен Дарина Андреевна

2026-08-30

Аннотация

Построена имитационная модель 25 конкурирующих потоков TCP/Reno, проходящих через узкое место 20 Мбит/с с активной очередью RED. Исследованы окно перегрузки, мгновенная и сглаженная длины очереди, вероятность раннего сброса и пропускная способность. Проведён параметрический эксперимент по числу потоков; реализация проверена автоматическими тестами.

Содержание

1. Цель и задание
2. Механизмы TCP/Reno и RED
3. Среда и организация проекта
4. Структура модели
5. Базовый эксперимент
6. Исследование числа потоков
7. Производные форматы и Jupyter
8. Проверка и управление версиями
9. Выводы
10. Источники

1. Цель и задание

Цель работы — исследовать совместную динамику алгоритма управления перегрузкой TCP/Reno и активного управления очередью RED в многопоточной сети.

Выполнены следующие задачи:

1. построена схема с 25 источниками, двумя маршрутизаторами и узким местом;
2. реализованы медленный старт, аддитивное увеличение и мультипликативное уменьшение окна TCP;
3. реализовано экспоненциальное сглаживание очереди и ранний сброс RED;
4. построены графики окна, очереди, вероятности сброса и throughput;
5. исследовано влияние количества конкурирующих потоков;
6. подготовлены данные, блокноты, документация и тесты.

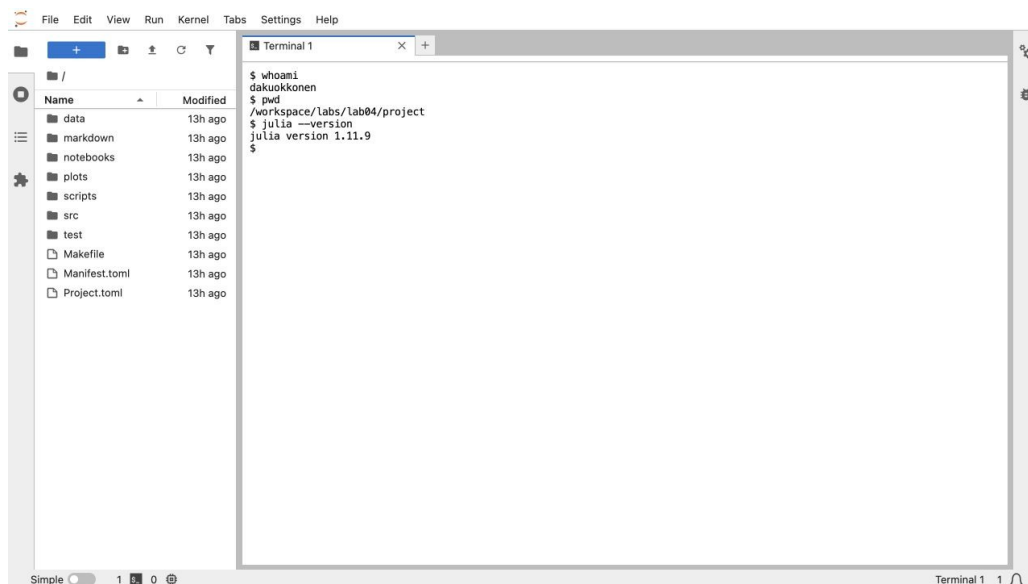


Рисунок 1: Проверка пользователя, версии Julia и рабочего каталога

2. Механизмы TCP/Reno и RED

TCP/Reno управляет окном перегрузки с wnd . При отсутствии потерь окно растёт: быстро в фазе slow start и примерно на один пакет за RTT в режиме congestion avoidance. Сигнал перегрузки вызывает мультипликативное уменьшение

$$cwnd \leftarrow \max\left(\frac{cwnd}{2}, 1\right). \quad (1)$$

RED принимает решение по сглаженной очереди

$$\dot{q}_{k+1} = (1 - w_q) \dot{q}_k + w_q q_k. \quad (2)$$

Вероятность раннего сброса задаётся кусочно-линейно:

$$p(\dot{q}) = \begin{cases} 0, & \dot{q} < q_{min}, \\ p_{max} \frac{\dot{q} - q_{min}}{q_{max} - q_{min}}, & q_{min} \leq \dot{q} < q_{max}, \\ p_{max}, & \dot{q} \geq q_{max}. \end{cases} \quad (3)$$

За счёт (Уравнение 2) алгоритм не реагирует на единичные всплески, но начинает сигнализировать о перегрузке до заполнения физического буфера.

Кусочно-линейная функция вероятности вынесена в общий модуль и непосредственно проверяется модульными тестами.

```
function red_probability(qavg::Real, red::REDParameters)
    qavg < red.qmin && return 0.0
    qavg >= red.qmax && return red.pmax
    return red.pmax *
        (qavg - red.qmin) / (red.qmax - red.qmin)
end
```

Листинг 1 — Вероятность раннего сброса в src/tcp_red.jl

3. Среда и организация проекта

Расчёты выполнены в Julia 1.11.9. Параметры сети, параметры RED, тип результата и имитационный цикл собраны в src/tcp_red.jl. Базовый сценарий сохраняет траекторию и сводку, а второй сценарий выполняет эксперимент по числу потоков.

Таблица 1: Структура вычислительной части лабораторной работы

Компонент	Назначение
src/tcp_red.jl	структуры параметров и имитация TCP/Reno с RED
scripts/01_tcp_red.jl	базовый эксперимент для 25 потоков
scripts/02_flow_scan.jl	исследование конфигураций от 5 до 40 потоков
scripts/tangle.jl	генерация Julia, QMD и IPYNB
test/runtests.jl	проверки функции RED и ограничений модели

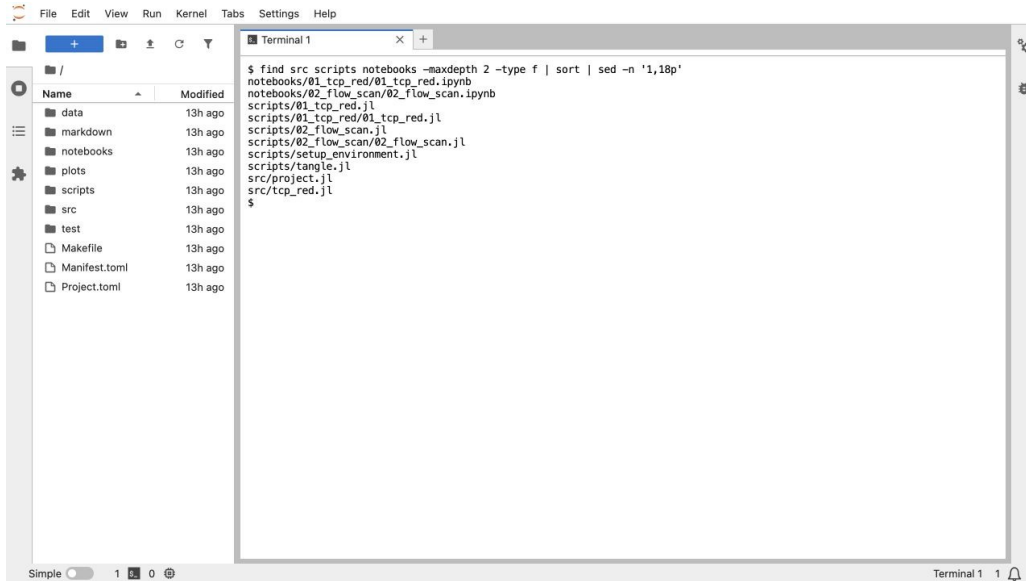


Рисунок 2: Состав Julia-проекта и закреплённые зависимости

4. Структура модели

Моделируется «гантелеобразная» топология: источники подключены к первому маршрутизатору быстрыми линиями, между маршрутизаторами находится узкое место, после второго маршрутизатора трафик поступает получателям.

Таблица 2: Параметры базового эксперимента

Параметр	Значение
Число TCP/Repo-потоков	25
Пропускная способность узкого места	20 Мбит/с
Задержка линии между маршрутизаторами	15 мс
Размер пакета	500 байт
Базовый RTT	118 мс
Буфер	300 пакетов
q_{min} / q_{max}	75 / 150 пакетов
w_q	0,002
p_{max}	0,1
Время моделирования	30 с

В каждый момент скорость потока оценивается как $c \cdot wnd / RTT$, где RTT включает задержку накопленной очереди. Принятые пакеты добавляются в буфер, после чего

обслуживаются с физической скоростью линии. Для каждого потока независимо разыгрывается сигнал RED.

```
rtt = propagation_rtt + queue / service_rate
sent = cwnd ./ rtt .* dt
p = red_probability(qavg, red)

if rand(rng) < 1 - (1 - p)^sent[i]
    cwnd[i] = max(cwnd[i] / 2, 1.0)
else
    cwnd[i] += dt / rtt
end
```

Листинг 2 — Обновление окна одного TCP-потока в src/tcp_red.jl

5. Базовый эксперимент

После переходного процесса средние показатели составили:

```
network = NetworkParameters(
    flows=25,
    capacity_bps=20e6,
    packet_bytes=500,
    propagation_rtt=0.118,
    duration=30.0,
    dt=0.002,
)
red = REDParameters(
    qmin=75.0,
    qmax=150.0,
    weight=0.002,
    pmax=0.1,
    limit=300.0,
)
result = simulate_tcp_red(; network, red, seed=202604)

trajectory = DataFrame(
    time=result.time,
    cwnd_first=result.cwnd_first,
    cwnd_mean=result.cwnd_mean,
    queue=result.queue,
    average_queue=result.average_queue,
    drop_probability=result.drop_probability,
    throughput_mbps=result.throughput_mbps,
)
```

Листинг 3 — Базовый сценарий в scripts/01_tcp_red.jl

Таблица 3: Результаты базового эксперимента

Показатель	Значение
Среднее окно одного потока	24,5234 пакета
Средняя мгновенная очередь	58,7519 пакета
Средняя EWMA-очередь	56,4558 пакета
Средняя пропускная способность	18,7654 Мбит/с
Число сигналов потери	446

Окно первого потока демонстрирует характерную пилообразную динамику AIMD: плавный рост чередуется с быстрым делением пополам. Усреднение по потокам сглаживает индивидуальные потери.

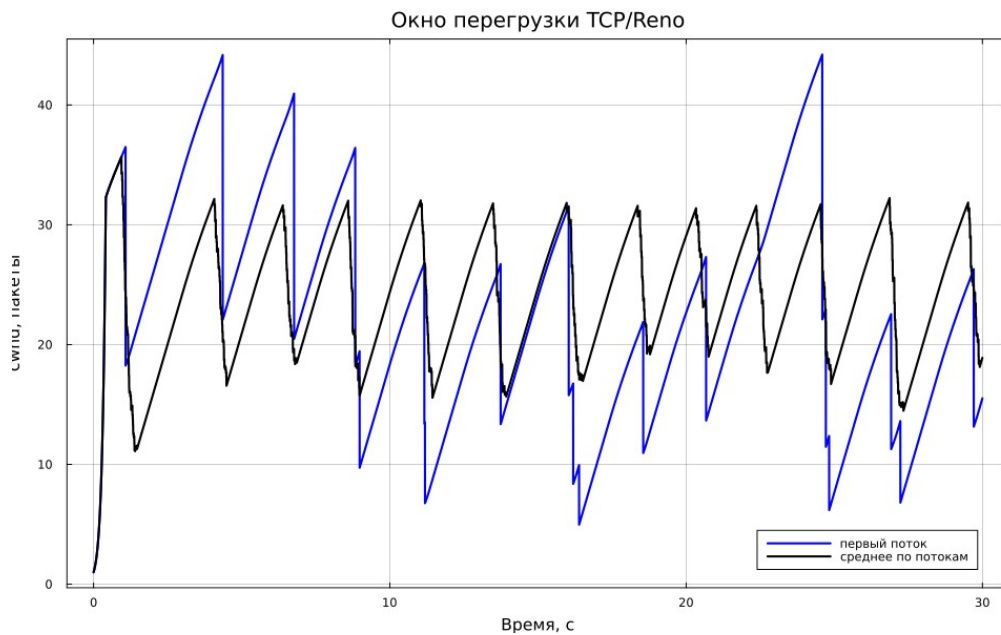


Рисунок 3: Окно перегрузки TCP/Reno

Мгновенная очередь изменяется быстрее EWMA. В базовом режиме среднее значение остаётся ниже q_{min} , однако всплески и накопленная сглаженная оценка периодически активируют ранний сброс.

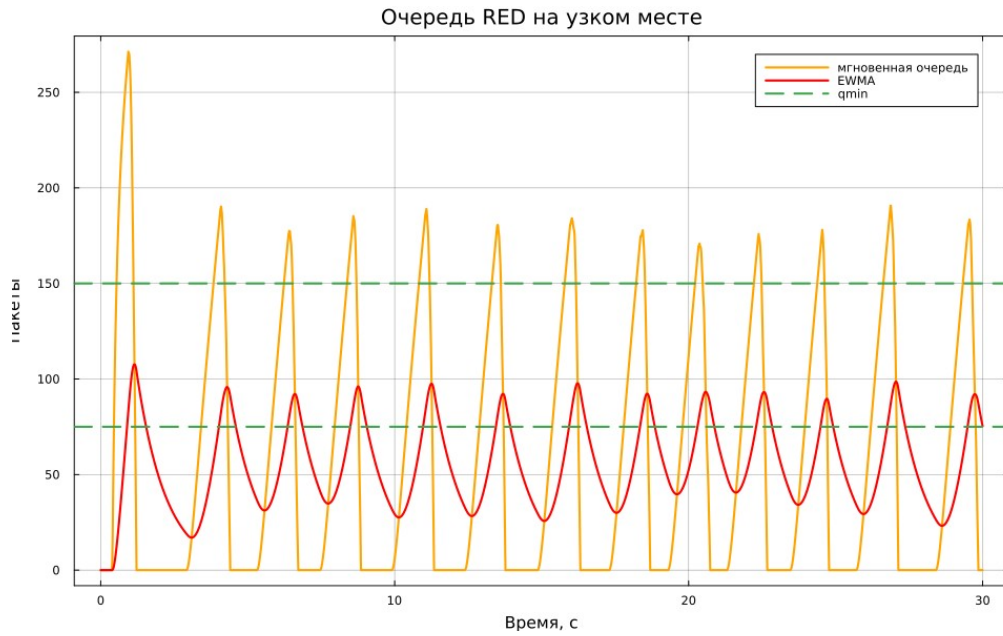


Рисунок 4: Мгновенная и сглаженная очереди RED

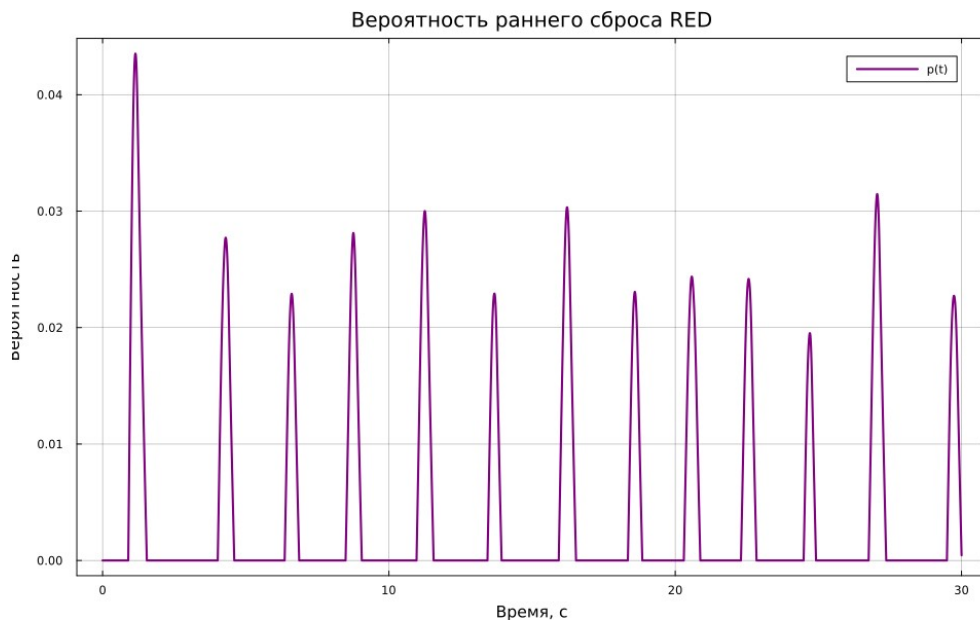


Рисунок 5: Вероятность раннего сброса

Линия большую часть времени используется почти полностью. Краткие провалы связаны с синхронной реакцией части потоков на сигналы перегрузки.

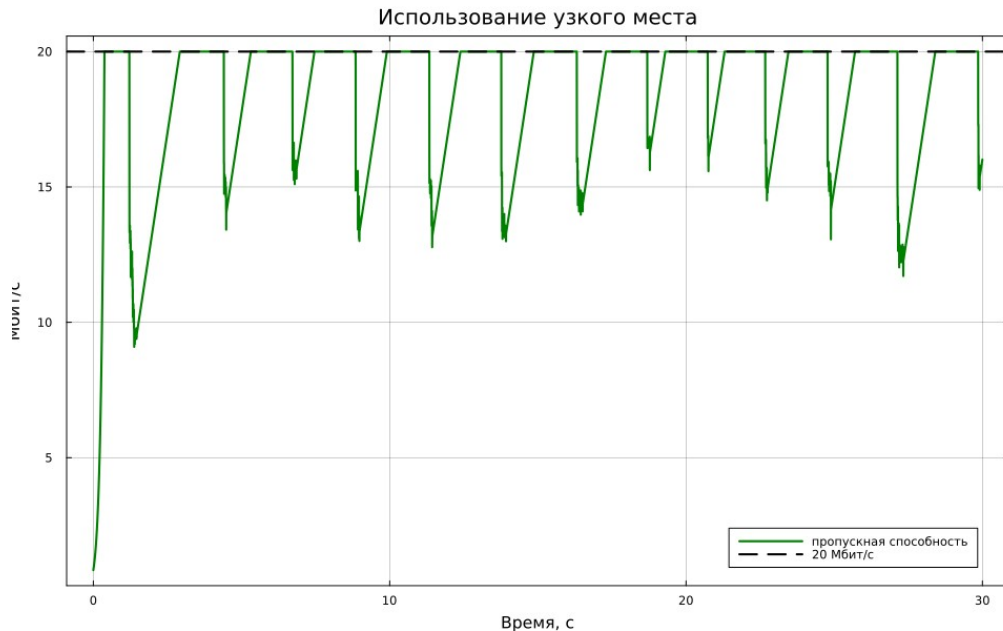


Рисунок 6: Пропускная способность узкого места

6. Исследование числа потоков

Число соединений изменялось от 5 до 40. Для каждого варианта использовалось отдельное фиксированное начальное число генератора.

```
rows = NamedTuple[]
for flows in [5, 10, 15, 20, 25, 30, 40]
    net = NetworkParameters(
        flows=flows,
        duration=30.0,
        dt=0.002,
    )
    result = simulate_tcp_red(
        network=net,
        red=REDParameters(),
        seed=202604 + flows,
    )
    warm = result.time .>= 10.0
    push!(rows, (
        flows=flows,
        throughput_mbps=mean(result.throughput_mbps[warm]),
        mean_queue=mean(result.queue[warm]),
        mean_cwnd=mean(result.cwnd_mean[warm]),
        loss_events=result.losses,
    ))
end
```

Листинг 4 — Эксперимент по числу потоков в scripts/02_flow_scan.jl

Таблица 4: Влияние числа конкурирующих потоков

Потоки	Throughput, Мбит/с	Средняя очередь	Среднее окно	Потери
5	19,7009	38,3260	124,0190	10
10	19,6181	45,3713	62,4388	53
15	18,5747	41,1950	39,3207	157
20	18,4500	47,9929	29,6584	277
25	18,4514	53,3248	23,9354	456
30	18,7217	63,9456	20,5946	646
40	18,7592	69,0914	15,6248	1063

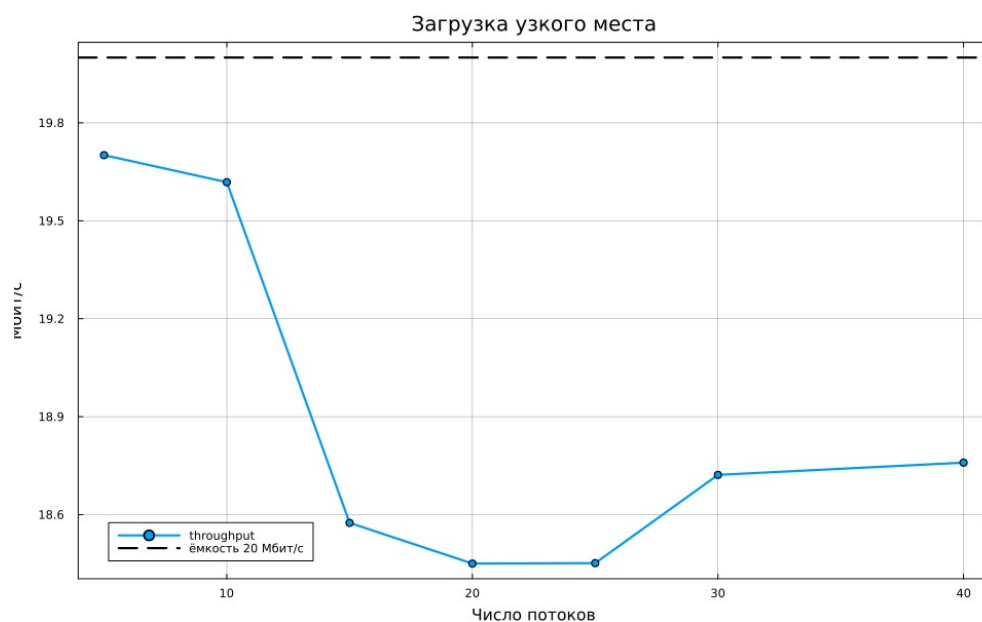


Рисунок 7: Использование узкого места при разном числе потоков

С ростом конкуренции индивидуальное окно уменьшается, а число сигналов потери растёт. RED удерживает среднюю очередь существенно ниже предела 300 пакетов.

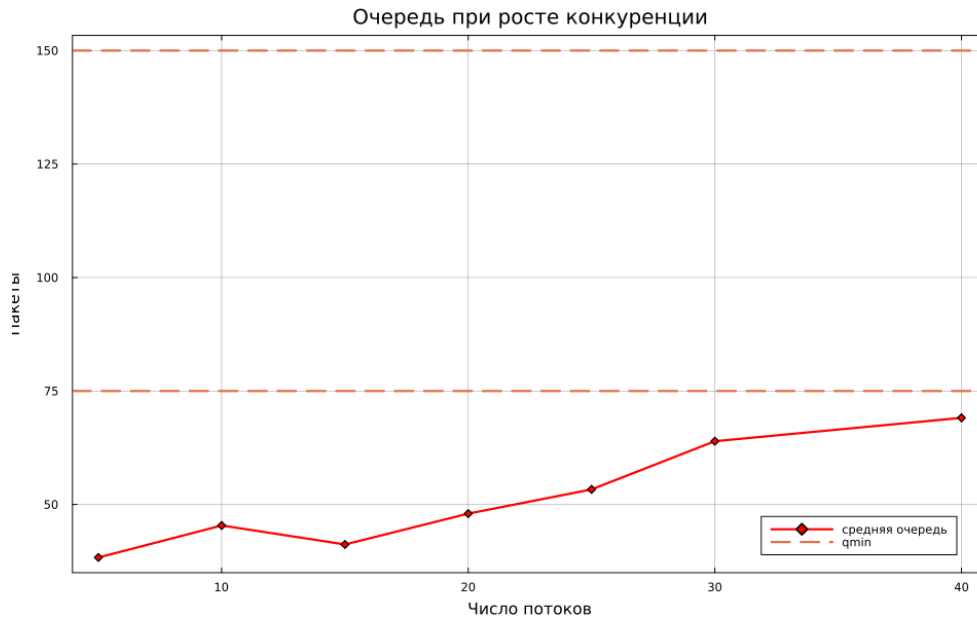


Рисунок 8: Средняя очередь при разном числе потоков

7. Производные форматы и Jupyter

Литературные исходники преобразуются в чистые Julia-файлы, QMD и IPYNB одним генератором. Блокноты выполняются с ядром Julia лабораторной работы и сохраняют фактические сводки базового и параметрического экспериментов.

```
function generate_formats(source::AbstractString)
    name = splitext(basename(source))[1]
    foreach(mkpath, (
        scriptsdir(name),
        projectdir("markdown", name),
        projectdir("notebooks", name),
    ))
    Literate.script(source, scriptsdir(name); name, credit=false)
    Literate.notebook(
        source, projectdir("notebooks", name);
        name, execute=false, credit=false,
    )
    Literate.markdown(
        source, projectdir("markdown", name);
        name, flavor=Literate.QuartoFlavor(), credit=false,
    )
end
```

Листинг 5 — Генерация производных форматов в scripts/tangle.jl

```
CSV.write(datadir(name, "summary.csv"), summary)
println("=== TCP/Reno + RED: базовый эксперимент ===")
println("Потоков: $(network.flows), узкое место: $(network.capacity_bps/1e6) Мбит/с")
show(summary; allrows=true, allcols=true); println()

default(fontfamily="DejaVu Sans", linewidth=2.2, framestyle=:box, gridalpha=0.22)
p1 = plot(trajecory.time, trajectory.cwnd_first; label="первый поток",
          xlabel="Время, с", ylabel="cwnd, пакеты", title="Окно перегрузки TCP/Reno",
          size=(1000,620), color=:blue)
plot!(p1, trajectory.time, trajectory.cwnd_mean; label="среднее по потокам", color=:black)
savefig(p1, plotsdir(name, "cwnd.png"))

p2 = plot(trajecory.time, trajectory.queue; label="мгновенная очередь",
          xlabel="Время, с", ylabel="Пакеты", title="Очередь RED на узком месте",
          size=(1000,620), color=:orange)
plot!(p2, trajectory.time, trajectory.average_queue; label="EWMA", color=:red)
hline!(p2, [red.qmin, red.qmax]; label=["qmin" "qmax"], linestyle=:dash)
savefig(p2, plotsdir(name, "red_queue.png"))

p3 = plot(trajecory.time, trajectory.drop_probability; label="p(t)",
          xlabel="Время, с", ylabel="Вероятность", title="Вероятность раннего сброса RED",
          size=(1000,620), color=:purple)
savefig(p3, plotsdir(name, "drop_probability.png"))

p4 = plot(trajecory.time, trajectory.throughput_mbps; label="пропускная способность",
          xlabel="Время, с", ylabel="Мбит/с", title="Использование узкого места",
          size=(1000,620), color=:green)
hline!(p4, [network.capacity_bps/1e6]; label="20 Мбит/с", linestyle=:dash, color=:black)
savefig(p4, plotsdir(name, "throughput.png"))

=== TCP/Reno + RED: базовый эксперимент ===
Потоков: 25, узкое место: 20.0 Мбит/с
5x2 DataFrame
Row | metric          value
```

Рисунок 9: Выполненный блокнот базового эксперимента TCP/Reno и RED

```
size=(1000,620), color=:red)
hline!(p2, [red.qmin, red.qmax]; label=["qmin" "qmax"], linestyle=:dash)
savefig(p2, plotsdir(name, "flow_queue.png"))

=== Сканирование числа TCP-потоков ===
7x5 DataFrame
Row | flows      throughput_mbps  mean_queue  mean_cwnd  loss_events
Int64 Float64      Float64      Float64      Int64
1   5         19.7009         38.326       124.019        10
2  10         19.6181         45.3713       62.4388         53
3  15         18.5747         41.195        39.3207        157
4  20         18.45         47.9929       29.6584        277
5  25         18.4514         53.3248       23.9354        456
6  30         18.7217         63.9456       20.5946        646
7  40         18.7592         69.0914       15.6248       1863

[1]: "/workspace/labs/lab04/project/plots/02_flow_scan/flow_queue.png"

С ростом числа потоков узкое место насыщается, а RED удерживает очередь в ограниченном диапазоне и
сигнализирует TCP до переполнения буфера.
```

Рисунок 10: Выполненный блокнот эксперимента по числу потоков

8. Проверка и управление версиями

Тесты проверяют кусочно-линейную функцию RED, границы очереди и вероятности, минимальное окно TCP, ограничение throughput физической ёмкостью, наличие доставленных пакетов и появление потерь в нагруженном режиме.

```

@testset "TCP/RED simulation" begin
    net = NetworkParameters(flows=25, duration=12.0, dt=0.002)
    red = REDParameters()
    r = simulate_tcp_red(network=net, red=red, seed=42)
    @test length(r.time) == length(r.queue)
    @test all(0 .<= r.queue .<= red.limit)
    @test all(r.cwnd_first .>= 1.0)
    @test all(0 .<= r.drop_probability .<= red.pmax)
    @test maximum(r.throughput_mbps) <=
        net.capacity_bps / 1e6 + 1e-9
    @test r.delivered_packets > 0
    @test r.losses > 0
end

```

Листинг 6 — Проверки имитационной модели в test/runtests.jl

Все 10 проверок пройдены. Дополнительно выполнены git diff --check и контроль состава отслеживаемых файлов.

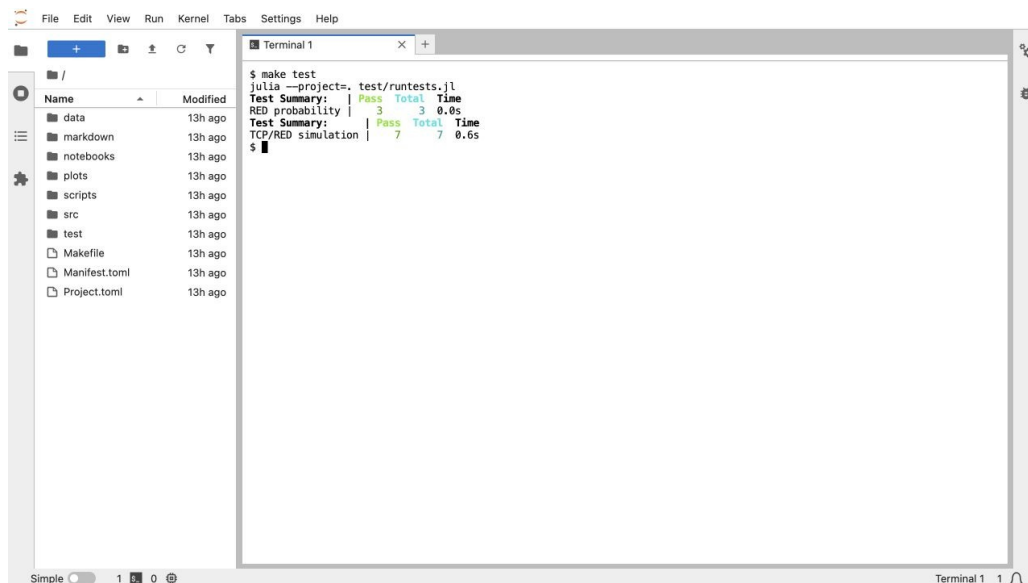


Рисунок 11: Результат выполнения автоматических проверок

9. Выводы

Построена воспроизводимая модель TCP/Reno с RED. В режиме 25 потоков узкое место обеспечивает в среднем 18,77 Мбит/с, а очередь остаётся значительно ниже физического лимита. Наблюдается ожидаемая пилообразная динамика окна AIMD. Параметрический эксперимент показал уменьшение индивидуального окна и рост числа сигналов перегрузки при увеличении конкуренции, при этом RED предотвращает неограниченное накопление пакетов.

10. Источники

1. Floyd S., Jacobson V. Random Early Detection Gateways for Congestion Avoidance. *IEEE/ACM Transactions on Networking*, 1993.
2. Stevens W. R. *TCP/IP Illustrated, Volume 1*. Addison-Wesley, 1994.
3. Julia Documentation. <https://docs.julialang.org/>
4. Quarto Documentation. <https://quarto.org/docs/>